

(Java)

OOP (Object Oriented Programming)

★ objects → entities in the real world
classes → group of these entities
↳ blueprint of the object

class → attributes (properties)
→ functions (behaviours)

4 major
pillars of
OOPs:-

- ① Encapsulation
- ② Inheritance
- ③ Abstraction
- ④ Polymorphism

* Access Modifiers / Access Specifiers:-

- 1.) Private → only within class
- 2.) Default → within class & Package
- 3.) Protected → within class, package & outside package
by subclass only
- 4.) Public → Accessible everywhere

* Getters & Setters :-

Get: to return the value

Set: to modify the value

[this:- this keyword is used to refer to the current object]

* Encapsulation:-

Encapsulation is defined as the wrapping up of data & methods under a single unit. It also implements data hiding.

↳ access specifiers

Ex:- class

* Constructors :-

Constructor is a special method which is invoked automatically at the time of object creation.
→ constructors have the same name as class or structure.

~~Constructors have the same name as class or structure.~~

- Constructors don't have a return type. (Not even void)
- Constructors are only called once, at object creation.
- Memory allocation happens when constructor is called.

~~Default constructors~~

~~Parameterized constructors~~

~~Copy constructors~~

* Java executes default constructor when we don't explicitly define constructors. (But only when you have no constructors in the class)

* Types of Constructors :-

1.) Non-parameterized constructor → doesn't take any parameter,
Ex:- `Student () {`
 ...
 }

2.) Parameterized constructor → takes parameters.
Ex:- `Student (String name) {`
 ...
 }

* You can create multiple constructors in a class.

★ Constructor overloading :- suitable constructors execute.

→ Example of polymorphism (multiple forms).

③ Copy constructors :-

→ by default in C++ but not in Java.

→ copy all properties of one object into another.

Ex: `Student (Student s) {`

`}`

→ object is passed

★ Issue in Copy Constructor :-

when there is an array which is being copied ideally the values modified after copying should not reflect but it does, bcz arrays use references.

★ Shallow & Deep Copy :-

★ Shallow → references are copied (changes reflect)

• Deep → new, created structures (changes don't reflect)

* Destructors :-

special functions which remove objects from the memory (in C++)

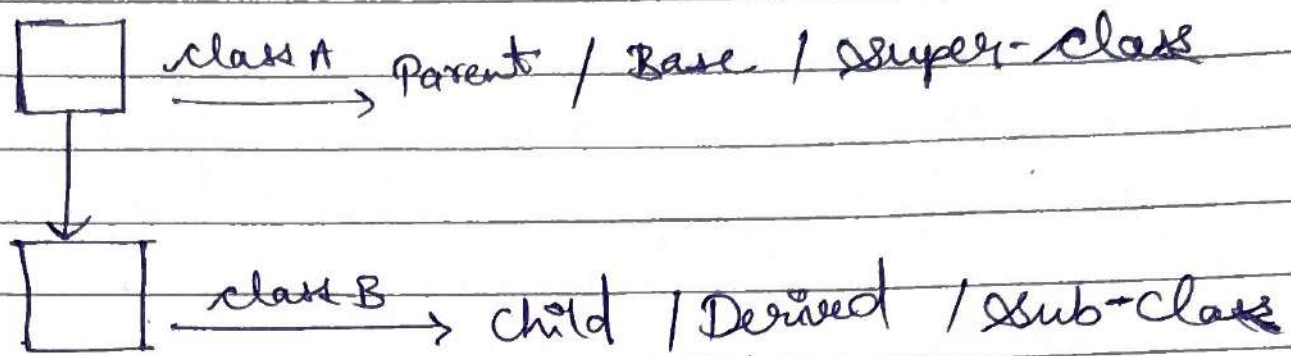
But in Java there is garbage collector :-

It ensures that if an object is not being used further, deletes them automatically.

P.T.O.

* Inheritance :-

Inheritance is when properties & methods of base class are passed on to a derived class



extends → Keyword

ex:- class Animal {

 - - -

class fish extends Animal {

}

* Types of Inheritance :-

① Single level inheritance -
one base class



one derived class

② Multi-level inheritance -

A Base class ~~parent class~~

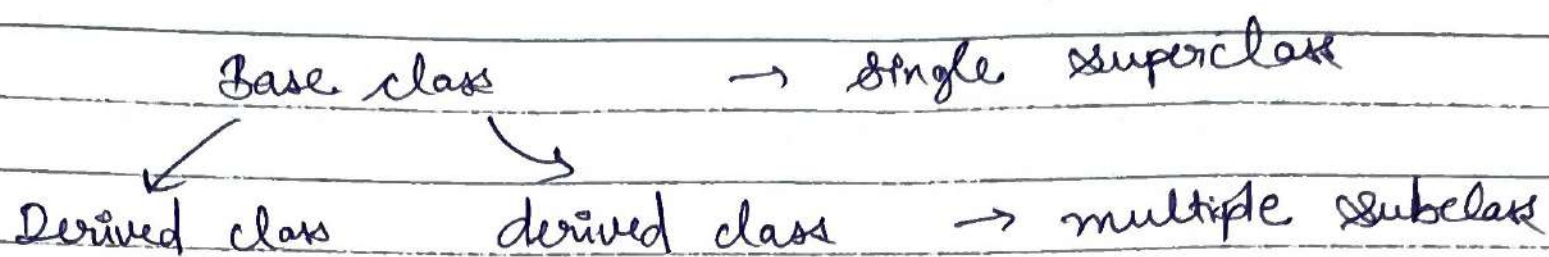


B Derived class ~~parent class of C~~



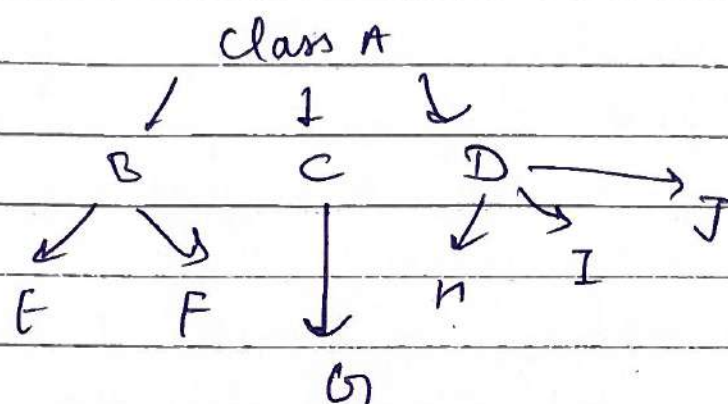
C Derived class → inherits both A & B

③ Hierarchical inheritance -

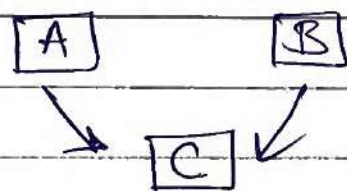


④ Hybrid inheritance - combination of two or more types of inheritance.

ex:-



* Java doesn't support multiple inheritance but C++ does



In Java we can indirectly implement this using interfaces.

* Polymorphism:-

many forms

When we try to achieve the same thing in multiple forms.

★ Types of Polymorphism:-

① Compile time polymorphism (static)
Ex:- Method Overloading

② Runtime polymorphism (dynamic)
Ex:- Method Overriding

★ Method Overloading → Multiple functions with the same name but different parameters.

★ return type doesn't affect method overloading

★ Method Overriding → Parent & child classes both contain the same function with a different definition.

Object of child class → calls child class fn

Object of Parent class → calls parent class fn

Ex:- class Animal {
 void eat() {-}

 }
 class Dog extends Animal {
 void eat() {-} void bark() {-}

Animal a = new Animal(); → valid

Animal a = new Dog(); → also valid (Wow!)

reference type

object type

→ This is called upcasting

Animal a = new Dog();

→ Here ~~the~~ a can access
but can't access bark() bcz reference
type = Animal

Basically:-

→ what you can access is decided by the reference type
→ what implementation runs is decided by the object type.

Animal a = new Dog(); a.eat();

→ Here compiler checks does animal have eat(),
Yes compilation successful.

→ later at run-time JVM choose which eat() to execute based on object type.

that's why method overriding is an example of Runtime Polymorphism.

* Packages in Java:-

Package is a group of similar types of classes, interfaces & sub-packages.

* Two types of packages:-
① In-built
→ ex:- util

② User defined
↳ package name
write at start of file.

* Abstraction:- Hiding all the unnecessary details & showing only the important parts to the user.
→ implemented using Abstract classes & Interfaces.

* Abstract Class:-

- abstract keyword
But abstract class 1 {
... }
}

functions can also
be abstract
↳
abstract void sum() {
...
}

- Cannot create an instance (object) of abstract class
- Can have abstract / non-abstract methods
- Can have constructors.
- But if a class have ~~non~~ abstract method it must be abstract class.
- Abstract class can be inherited by both abstract / non-abstract sub-classes
- It is necessary for child class to implement the abstract methods present in the parent abstract class → (only if child class is non-abstract)
- Abstract classes can have constructors because their constructors are used to initialize the inherited part of subclass objects.
- Although an abstract class cannot be instantiated directly, its constructor is invoked when a concrete subclass object is created.

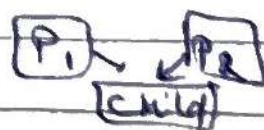
* a constructor is not for creating the object.
A constructor is for initializing the object's fields.

* Interfaces:-

- Interface is a blueprint of a class.

* Need of interfaces:-

- ① to implement multiple inheritance
- ② to implement total abstraction



• Keyword :- Interface

• Keyword for extending interfaces : $\rightarrow$ implements

- $\rightarrow$ All methods are public, abstract & without implementation
- $\rightarrow$ Used to achieve abstraction
- $\rightarrow$ Variables in the interface are final, public & static.
- $\rightarrow$ There are no interface objects. (bcz if all functions are abstract, there is no implementation so object creation would be useless).
- $\rightarrow$ Every class implementing an interface & calling its functions must make those functions public bcz they are public in interface so a class which implements that interface ~~must~~ cannot make it less visible, it must follow interface rules.
- $\rightarrow$ A concrete class implementing interface must implement its all abstract methods. (if the class is abstract then it is not necessary).

(★ Java requires methods to belong to a class or record / enum / interface.)

* Static Keyword :-

static keyword in Java is used to share the same variable or method of a given class / interface.

- Properties, functions, Blocks, nested classes.

$\downarrow$
(. -)

But you can't make outer class static

★ Statements cannot be directly inside a class.
Inside a class body, Java only allows variables, methods, constructors, block or nested class.

★ block → a group of statements enclosed in {}

★ initializer block → An initializer block is a block that is written directly inside a class, not inside method or constructor.

→ This block automatically runs whenever an object is created.

★ constructor runs after initializer block

Q. Why do initializer blocks exist?

→ for common initialization code shared by all constructors

→ Running setup code before constructors.

• Only initializer blocks can be static / non-static

→ An initializer block can only contain statements.

★ Super Keyword :-

Super keyword is used to refer immediate parent class object.

→ to access parent's properties.

→ to access parent's functions.

→ to access parent's constructors.

★ super is used when you specially want the parent's version of something.

Java automatically inserts super() only when possible.

But whenever you need:

A parametrized parent constructor,
a hidden parent variable,
or an overridden parent method,
you must write `super` yourself.

★ `private` & `protected` are not allowed in front of class